

Algorithm Performance Measurement and Benchmarking Tool

Lab Assignment 2: Final Report

Field	Detail
School	School of Engineering & Technology
Department	SOET
Program / Semester	BCA (AI&DS), Semester V, Session 2026-27
Course	ENCA351 Design and Analysis of Algorithms Lab
Faculty	Dr. Aarti
Student name	Shreyansh Vaibhaw
Roll number	2401201094
GitHub repository	algorithm-performance-tool-shreyansh
Reference run	2026-09-28 11:22:50, CPython 3.12.10, Windows-11-10.0.26200-SP0

1. Introduction

1.1 Importance of algorithm analysis

An algorithm that feels instant on a test file can stall on production data. Algorithm analysis exists to predict that before it happens. It describes how running time and memory grow with the input size n , in classes such as $O(\log n)$, $O(n)$ and $O(n^2)$, and it does so without reference to a particular machine or language. That independence is the point: the class tells us that an $O(n^2)$ routine will take about a hundred times longer when its input grows tenfold, on any computer, forever.

The same independence is also the limitation. Big-O notation hides constant factors and machine effects on purpose, so it cannot say how many milliseconds a call will take on a given laptop, whether a function that is $O(n)$ in theory has been implemented that way, or whether an assumption baked into the analysis, such as constant-time arithmetic, actually holds for the data at hand.

1.2 Need for performance measurement

Measurement fills that gap. Running the code at several input sizes and recording time, memory and the number of basic operations shows the constants theory hides, confirms or contradicts the predicted growth, and exposes hidden costs. In this project one such cost turned out to matter: Python integers have arbitrary precision, so the n multiplications inside a factorial do not take constant time once the running product has hundreds of digits.

Measuring well is harder than it looks. A timer has finite resolution, one run of a microsecond function is mostly noise, garbage collection can fire in the middle of a measurement, and a memory profiler changes the program it profiles. Part of the work in this assignment was getting those details right and documenting them, so that the numbers in this report mean what they appear to mean.

1.3 Objectives and scope

- Build a Python tool that executes a chosen algorithm or code snippet at a chosen input size and reports execution time, memory usage and operation counts.
- Compare linear search with binary search on datasets of 100 to 50,000 records (Task 2).
- Compare four program structures: single loop, nested loop, recursive factorial and iterative factorial (Task 3).
- Automate the experiments across the assignment's input sizes and record every measurement (Task 4).
- Chart the results, fit the observed growth trend and set it against the theoretical complexity (Tasks 5 and 6).
- Discuss which implementations perform better, how input size affects them, and the time-space trade-offs involved (Task 7).

The tool is written in Python 3 with Streamlit for the web interface, pandas for tables, matplotlib for charts, tracemalloc and memory_profiler for memory, and Jupyter for the notebook. All code, results, charts and this report are in the GitHub repository named on the cover page.

2. Tool design and measurement method

2.1 Architecture

The project is split into small modules with one job each. The Streamlit app and the notebook are thin layers over the same functions, so an experiment run from the command line, from the notebook and from the web page produces the same kind of record.

File	Responsibility
search_analysis.py	Linear and binary search that count key comparisons; dataset generation with a fixed seed; worst-case key selection.
code_benchmark.py	The four code snippets. Each returns its value and the number of basic operations implied by its structure.
benchmark_engine.py	Timing, memory measurement, the Measurement record, the automated suite across input sizes, CSV output.
complexity_analysis.py	Theoretical table, power-law fit of time against n (the observed growth exponent), theory-versus-observed table.
visualization.py	matplotlib charts with one fixed colour per algorithm, and plain-language trend summaries.
app.py	Streamlit web app with one page per task.
run_benchmarks.py	One command that runs the suite and writes results/, graphs/ and the complexity table.
project_notebook.ipynb	The whole study as an executed Jupyter notebook.

2.2 Measuring execution time

- **Timer.** `time.perf_counter()` is read immediately before and after the call. It is the highest-resolution monotonic clock Python offers.
- **Repetition.** Fast functions are called repeatedly until at least five samples covering 50 ms of total time exist; the median is reported. A 3 s budget stops the loop early, so a function that takes minutes runs once. Binary search was timed 1,000 times per size, the largest nested loop once.
- **Garbage collection** is paused while timing, as Python's own `timeit` module does, so a collection cycle cannot land inside a sample.
- **Argument construction** happens before the clock starts. Generating a 50,000-element dataset is not part of a search's time.

2.3 Measuring memory

Memory is measured with tracemalloc, which hooks every allocation the Python interpreter makes and reports the peak amount of heap memory in use. The engine records the peak extra memory allocated during one call, relative to the level just before it, so the numbers are the algorithm's own footprint plus its returned result.

Two decisions follow from how tracemalloc works.

- **Timing and tracing are separate runs.** Tracing every allocation slowed the loop snippets by 24x to 60x on the test machine. A time measured while tracing describes the profiler, not the algorithm, so the engine times first without tracing and then traces a second, untimed run.
- **Traces that would take hours are skipped.** The engine multiplies the untraced time by the measured overhead factor to predict the traced time. When that exceeds a 30 s budget it records "not measured" and the reason. This happened for the nested loop at 10,000 and 50,000 iterations: tracing the 2.5 billion allocations of the largest case would have taken about three hours. Its $O(1)$ memory is confirmed at 100 and 1,000 iterations, where the trace is affordable.

tracemalloc sees Python heap objects, not the interpreter's call frames, which live in a separate stack area. For the recursive factorial this matters: the $O(n)$ stack shows up indirectly, through the one integer argument each open frame keeps alive, so the true cost of recursion is larger than the traced number. The tool also offers memory_profiler as a second backend. It samples the whole process's resident set size through psutil at page granularity, which is what a task manager would show, and it reports zero growth for most of these small algorithms.

2.4 Counting operations

- Linear search: one comparison per element examined.
- Binary search: one comparison per probe of the middle element. A probe needs an equality test and an ordering test in Python, but algorithmically it is one three-way comparison, the usual textbook convention.
- Loops: one operation per iteration of the innermost loop (n for the single loop, n squared for the nested loop).
- Factorials: n recursive calls, or $n - 1$ multiplications. These counts follow exactly from the structure of the snippet, so they are derived from n rather than counted with an extra variable that would slow the timed code.

2.5 Experimental setup

Item	Setting
Machine	Intel64 Family 6 Model 142 Stepping 12, GenuineIntel, 8 logical CPUs
Operating system	Windows-11-10.0.26200-SP0
Python	CPython 3.12.10
Search datasets	Distinct integers, sorted, seed 42; sizes 100, 1,000, 10,000 and 50,000
Search key	-1, which is never present: the worst case for both algorithms

Item	Setting
Factorial inputs	n = 100, 500 and 1,000 (the recursion limit is raised for n = 1,000)
Loop inputs	n = 100, 1,000, 10,000 and 50,000 iterations
Command	python run_benchmarks.py, one process, no other load

3. Experimental results

All numbers below come from the reference run saved in results/benchmark_results.csv. Times are medians of the repeated runs described in section 2.2; memory is the tracemalloc peak from a separate run.

3.1 Search algorithms

Table 1 shows both searches on the same sorted datasets with an absent key. Linear search's time grew 970x while the dataset grew 500x; binary search's grew 1.4x.

n	Linear: time	Linear: comparisons	Linear: memory	Binary: time	Binary: comparisons	Binary: memory
100	10.5 us	100	288 bytes	3.6 us	6	288 bytes
1,000	196.7 us	1,000	376 bytes	3.4 us	9	316 bytes
10,000	1.66 ms	10,000	376 bytes	4.9 us	13	316 bytes
50,000	10.18 ms	50,000	376 bytes	5.0 us	15	316 bytes

Table 1. Search results for an absent key (worst case).

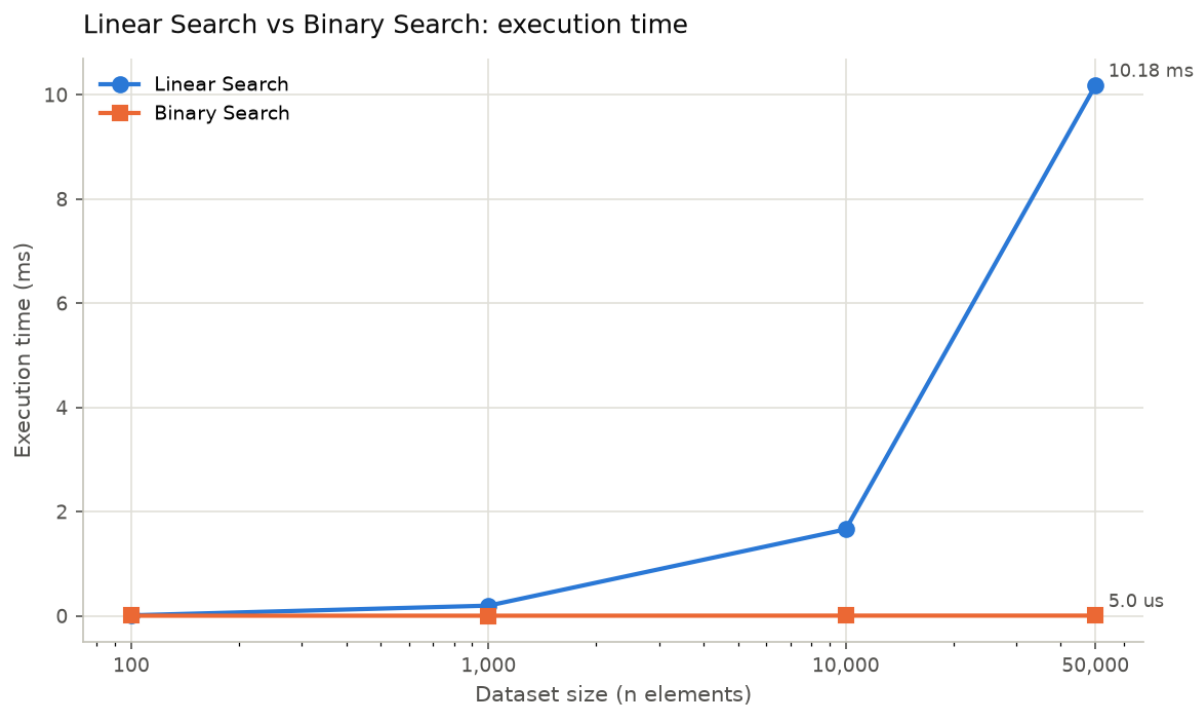


Figure 1. Dataset size against execution time, linear axes. Binary search is flat at a few microseconds.

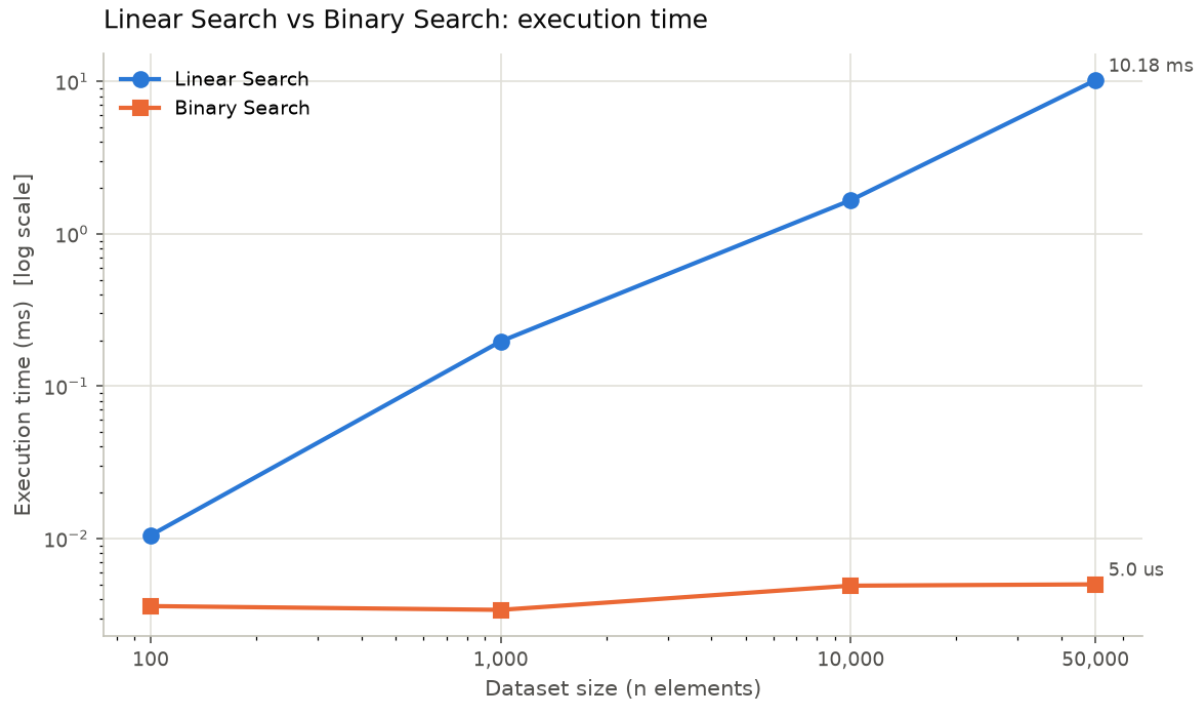


Figure 2. The same data with a logarithmic time axis, which makes both trends visible.

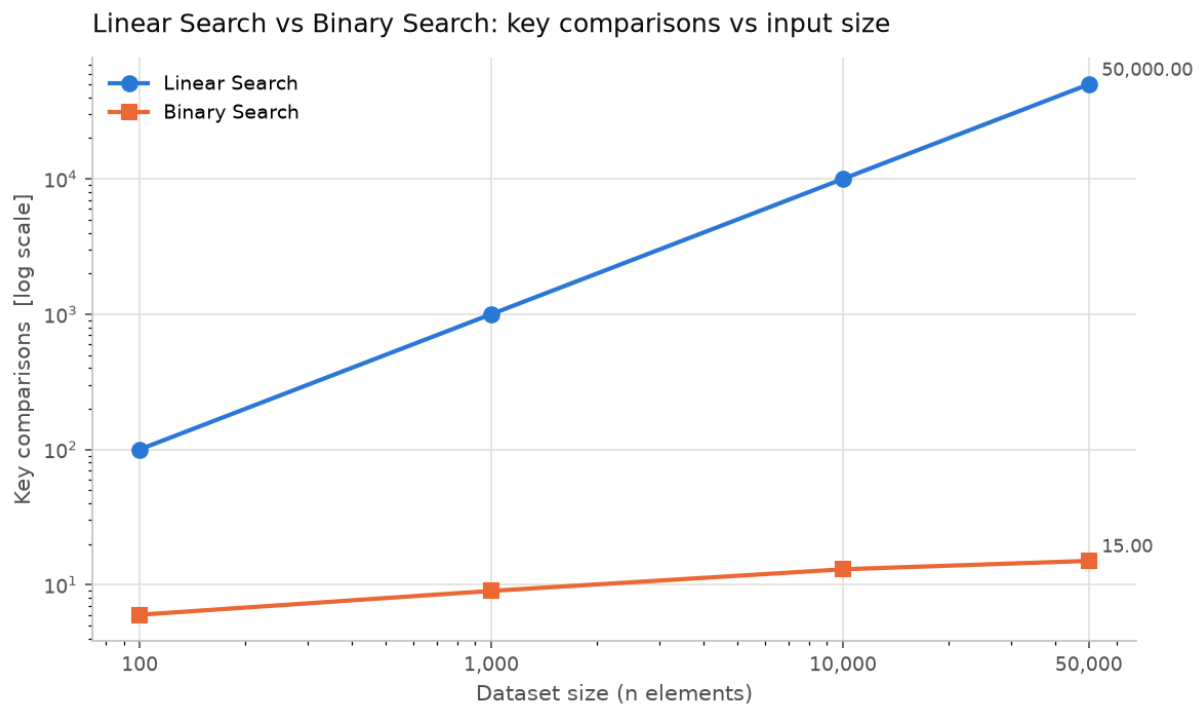


Figure 3. Key comparisons against dataset size. Linear search needs n ; binary search needs $\text{floor}(\log_2 n) + 1$.

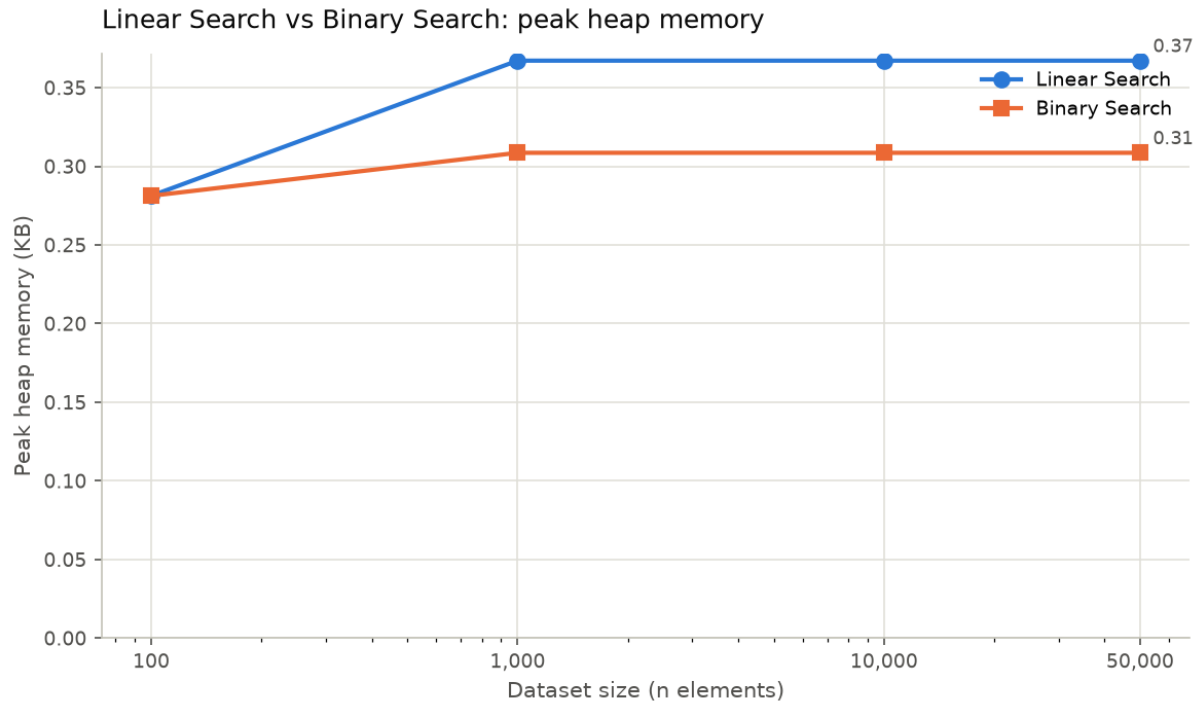


Figure 4. Peak heap memory. Both searches use a few hundred bytes regardless of n .

3.2 Factorial programs

Table 2 compares the two factorial implementations. At $n = 1,000$ the recursive version took 1.24x as long as the loop and peaked at 23.77 KB of heap against 2.39 KB.

n	Recursive: time	Recursive: memory	Iterative: time	Iterative: memory	Time ratio (rec / iter)
100	21.0 us	324 bytes	14.3 us	324 bytes	1.47x
500	228.2 us	8.14 KB	160.9 us	1.15 KB	1.42x
1,000	692.0 us	23.77 KB	558.5 us	2.39 KB	1.24x

Table 2. Recursive against iterative factorial.

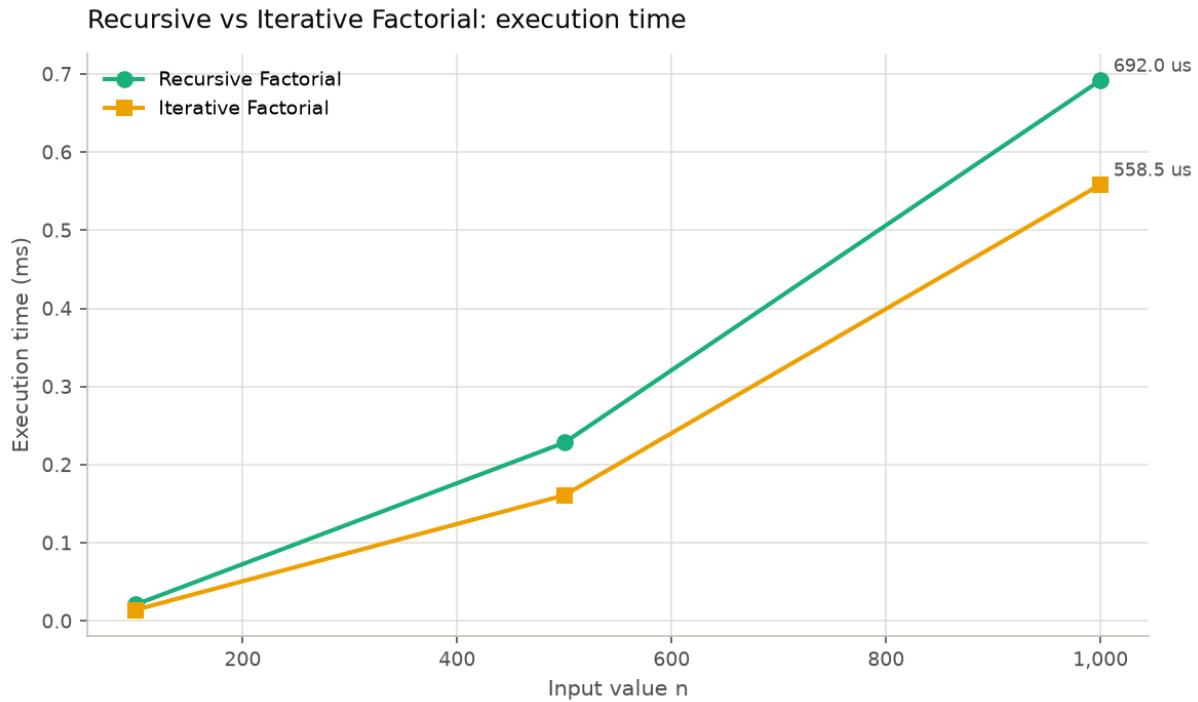


Figure 5. Input n against execution time for the two factorial versions.

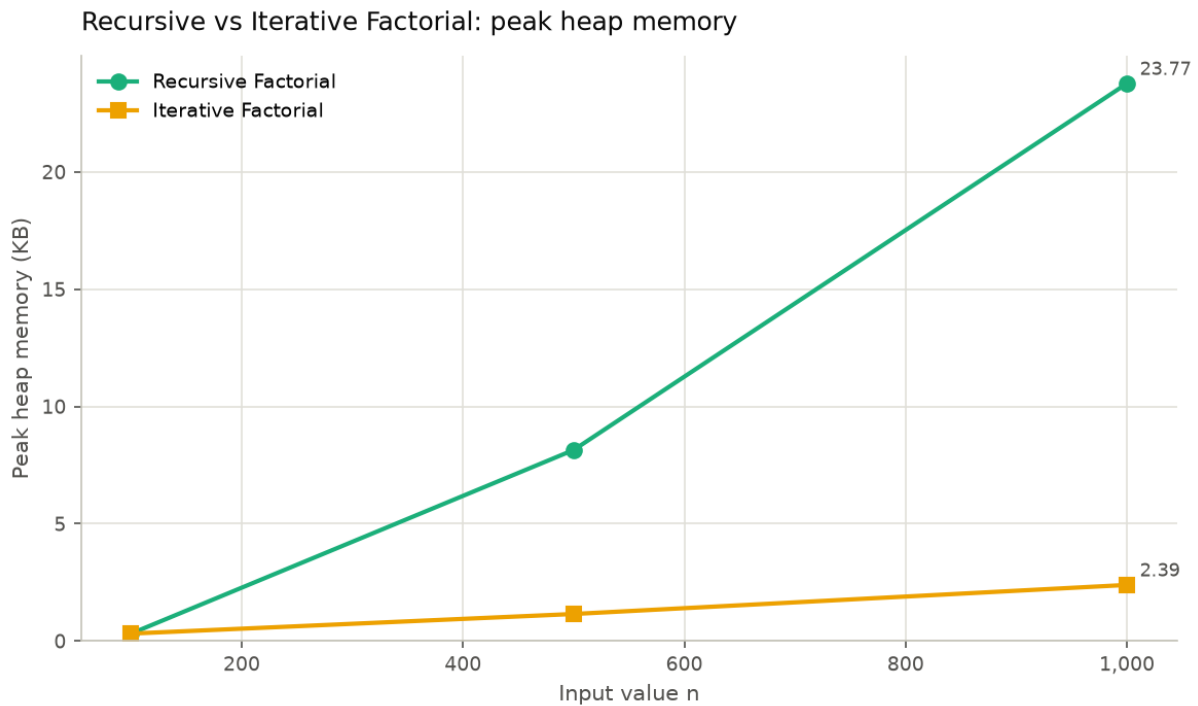


Figure 6. Peak heap memory. The recursive version grows linearly with n ; the loop's growth is the size of $n!$ itself.

3.3 Loop-based programs

Table 3 compares the single and nested loops. At $n = 50,000$ the nested loop performed 2,500,000,000 inner iterations and took 328.99 s, against 4.44 ms for the single loop: a ratio of 74,068x.

n	Single: time	Single: memory	Nested: time	Nested: memory	Nested: inner iterations
100	7.0 us	256 bytes	691.1 us	256 bytes	10,000
1,000	78.4 us	288 bytes	80.21 ms	320 bytes	1,000,000
10,000	866.2 us	288 bytes	8.85 s	not traced	100,000,000
50,000	4.44 ms	292 bytes	328.99 s	not traced	2,500,000,000

Table 3. Single against nested loop traversal. Memory for the nested loop at 10,000 and 50,000 was not traced (section 2.3).

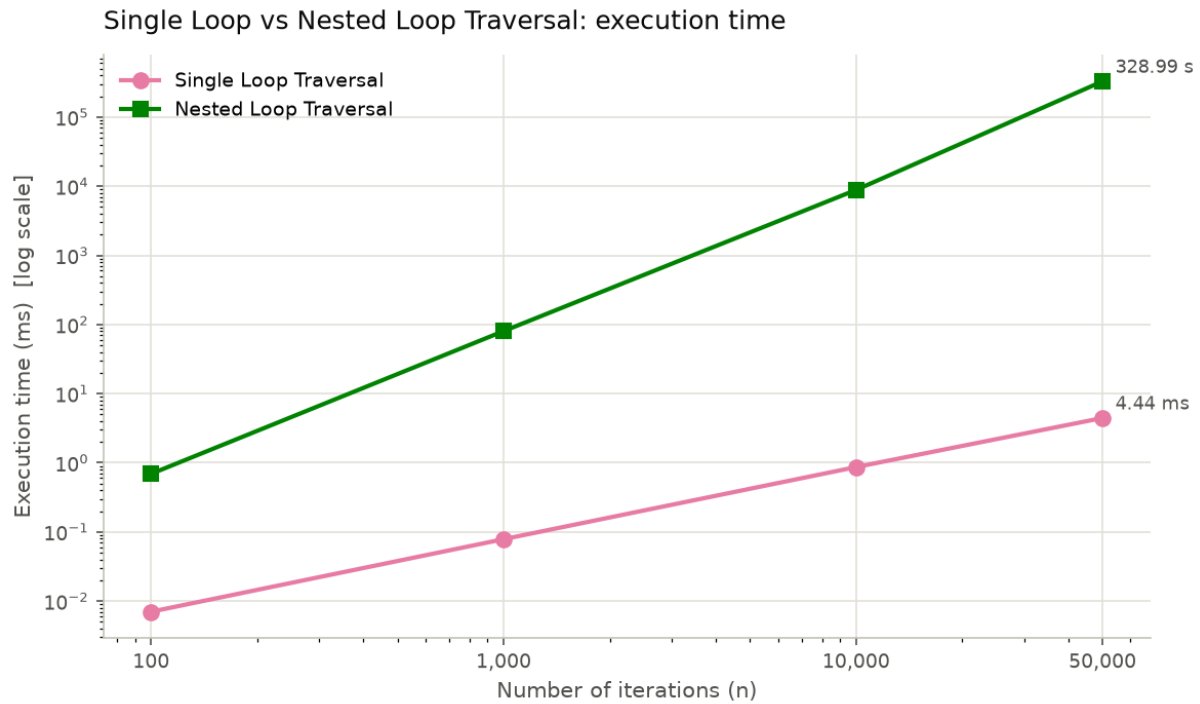


Figure 7. Iterations against execution time, logarithmic time axis. The nested loop's line is twice as steep.

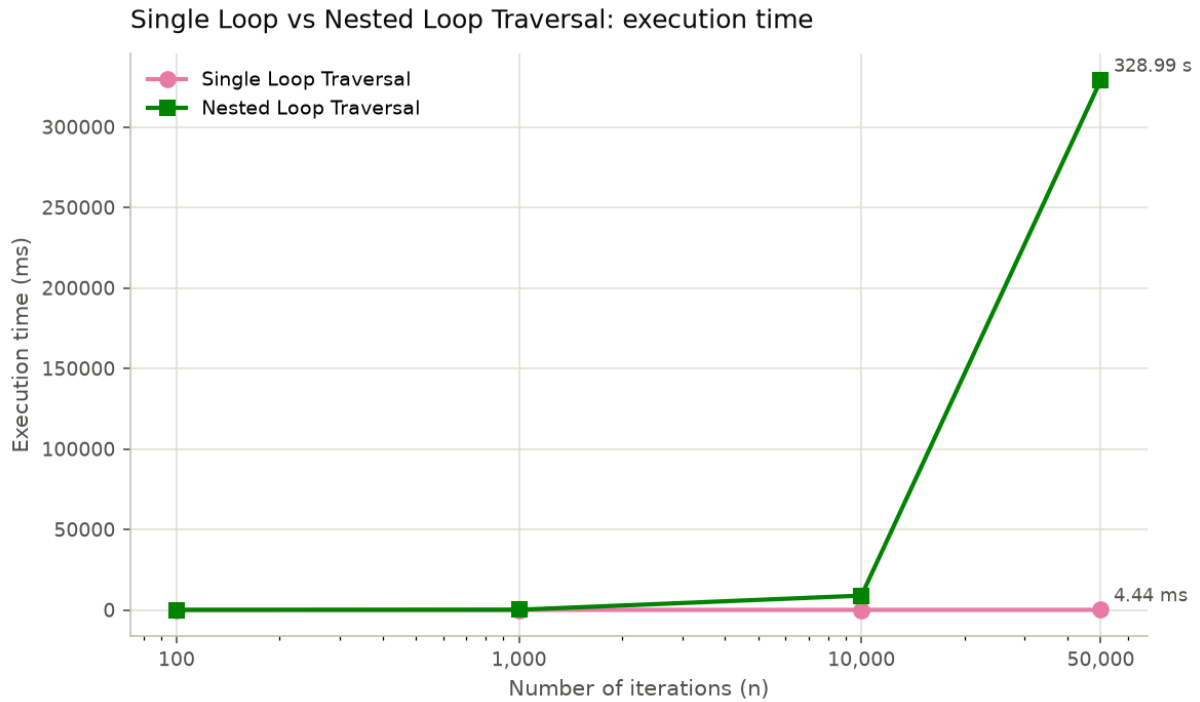
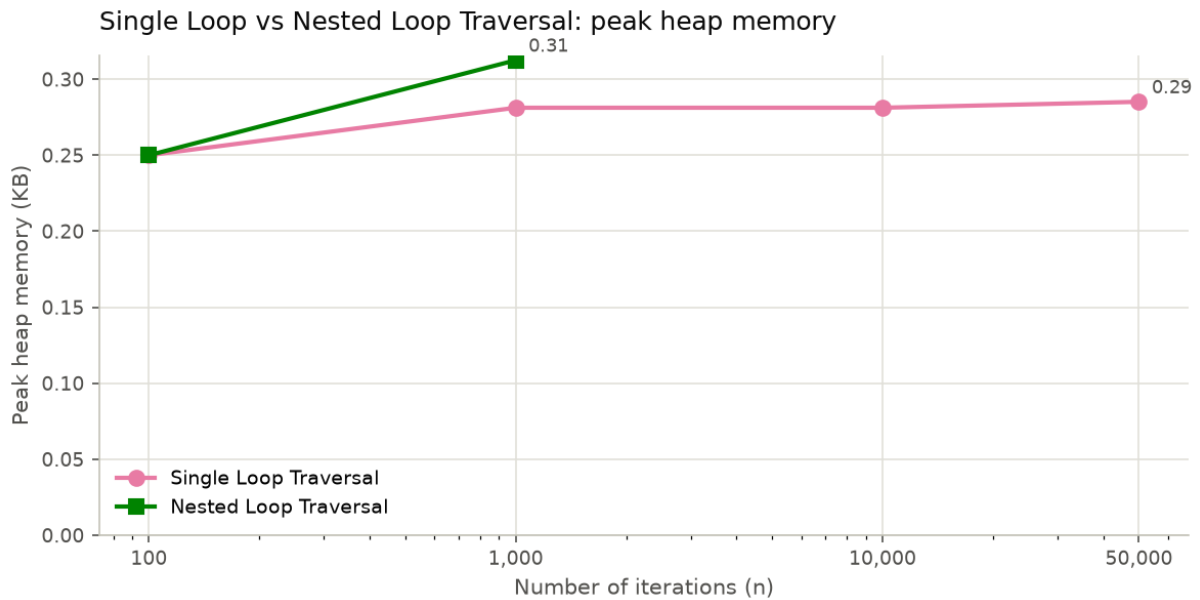


Figure 8. The same data on linear axes; the single loop is invisible next to the nested loop.



Memory not traced at $n = 10,000, 50,000$: tracemalloc would slow the run past the time budget.

Figure 9. Peak heap memory for the loops at the sizes where tracing was affordable.

3.4 All algorithms together

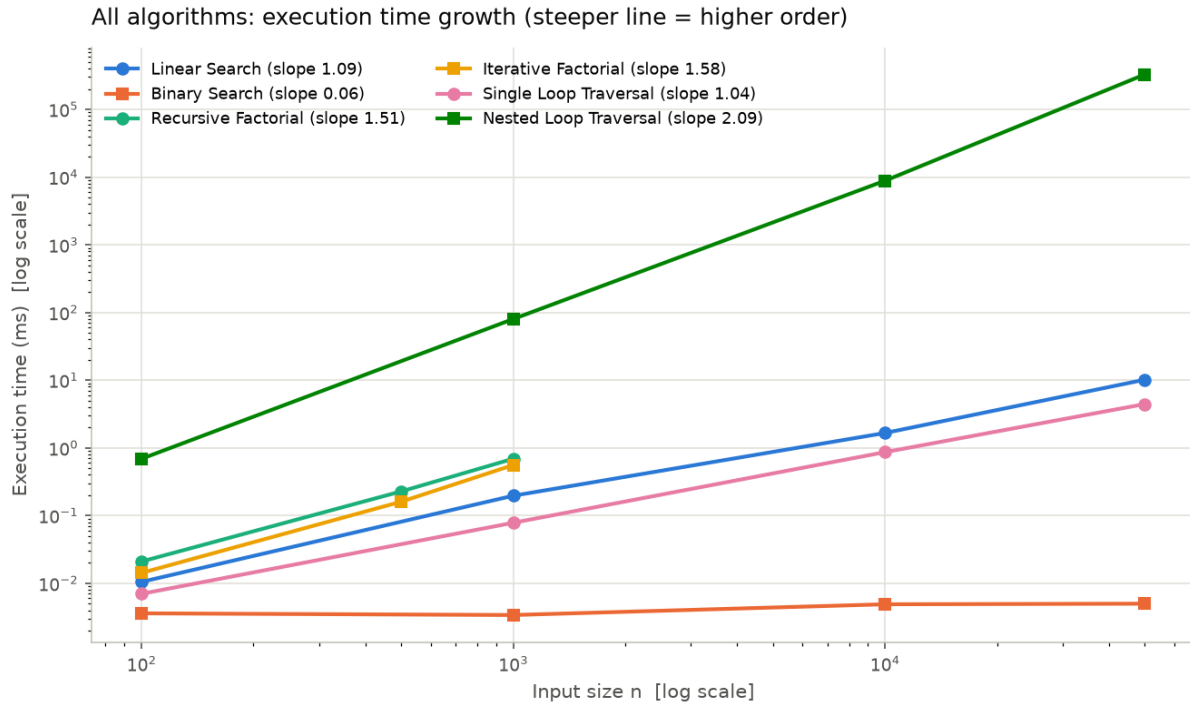


Figure 10. Every algorithm on log-log axes. A power law is a straight line whose slope is the growth exponent.

3.5 Screenshots of the tool

The Streamlit application, started with `streamlit run app.py`. Each page corresponds to one task of the assignment.

Benchmarking Tool

Module

- ☒ Overview
- ☐ Search Analysis
- ☐ Code Benchmarking
- ☐ Automated Benchmarking
- ☐ Visualization
- ☐ Complexity Analysis
- ☐ Comparative Study

Python 3.12.10 on Windows | 8 CPUs

ENCA351 Design and Analysis of Algorithms Lab, Lab Assignment 2

Shreyansh Vaibhaw, roll number 2401201094

Algorithm Performance Measurement and Benchmarking Tool

Pick an algorithm or a code snippet, choose an input size, run it, and read off the execution time, the memory it used and how many basic operations it performed. The automated module repeats this across a range of sizes so growth trends can be charted and compared with the theoretical complexity.

Search analysis

Linear and binary search on generated datasets of any size, with the number of key comparisons, execution time and memory for each run.

Code benchmarking

Single loop, nested loop, recursive factorial and iterative factorial, executed side by side with a comparison table and charts.

Automated study

Every algorithm across 100 to 50,000 elements, charts of size versus time, a complexity table and a written comparison.

How the measurements are made

- Execution time** comes from `time.perf_counter()` around the call. The call is repeated until there are at least five samples covering 50 ms, or a 3 s budget is spent, and the median is reported. Garbage collection is paused while timing, as `timeit` does.
- Memory** is the peak extra heap allocated during one separate run traced with `tracemalloc`. Timing and tracing are never done in the same run: tracing every allocation slows a tight Python loop 25x to 60x, so a traced run measures the profiler, not the code.
- Operations** are key comparisons for the searches and loop iterations, multiplications or recursive calls for the snippets.
- Dataset generation** is excluded from the clock. Only the algorithm is timed.
- If tracing a run is predicted to take longer than a budget (30 s by default) the memory pass is skipped and the reason is written into the results table.

This machine

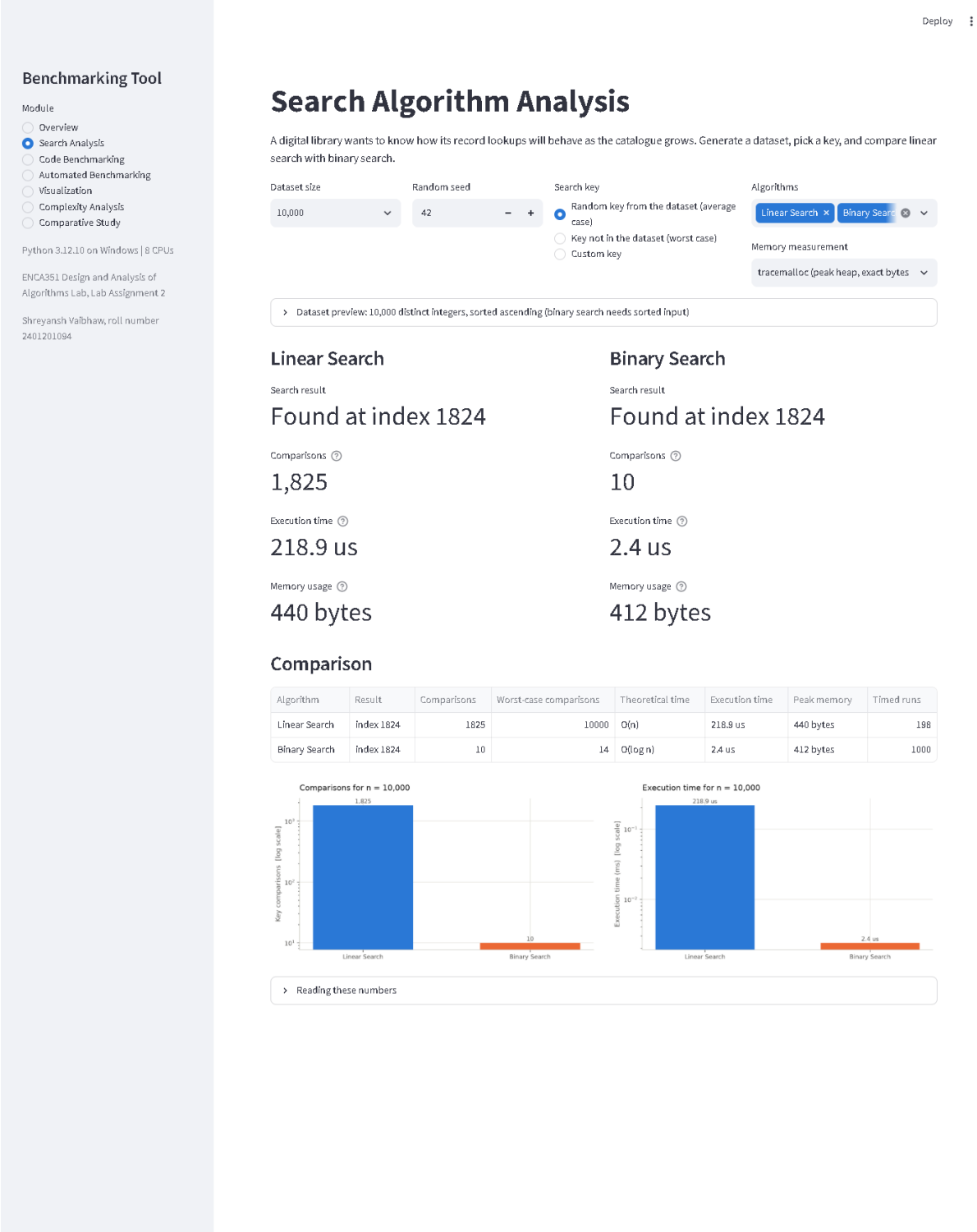
Setting	Value
python	3.12.10
implementation	CPython
os	Windows-11-10.0.26200-SP0
processor	Intel64 Family 6 Model 142 Stepping 12, GenuineIntel
cpu_count	8
timer	time.perf_counter
memory	tracemalloc peak heap (separate run)
recorded_at	2026-09-28 12:33:01

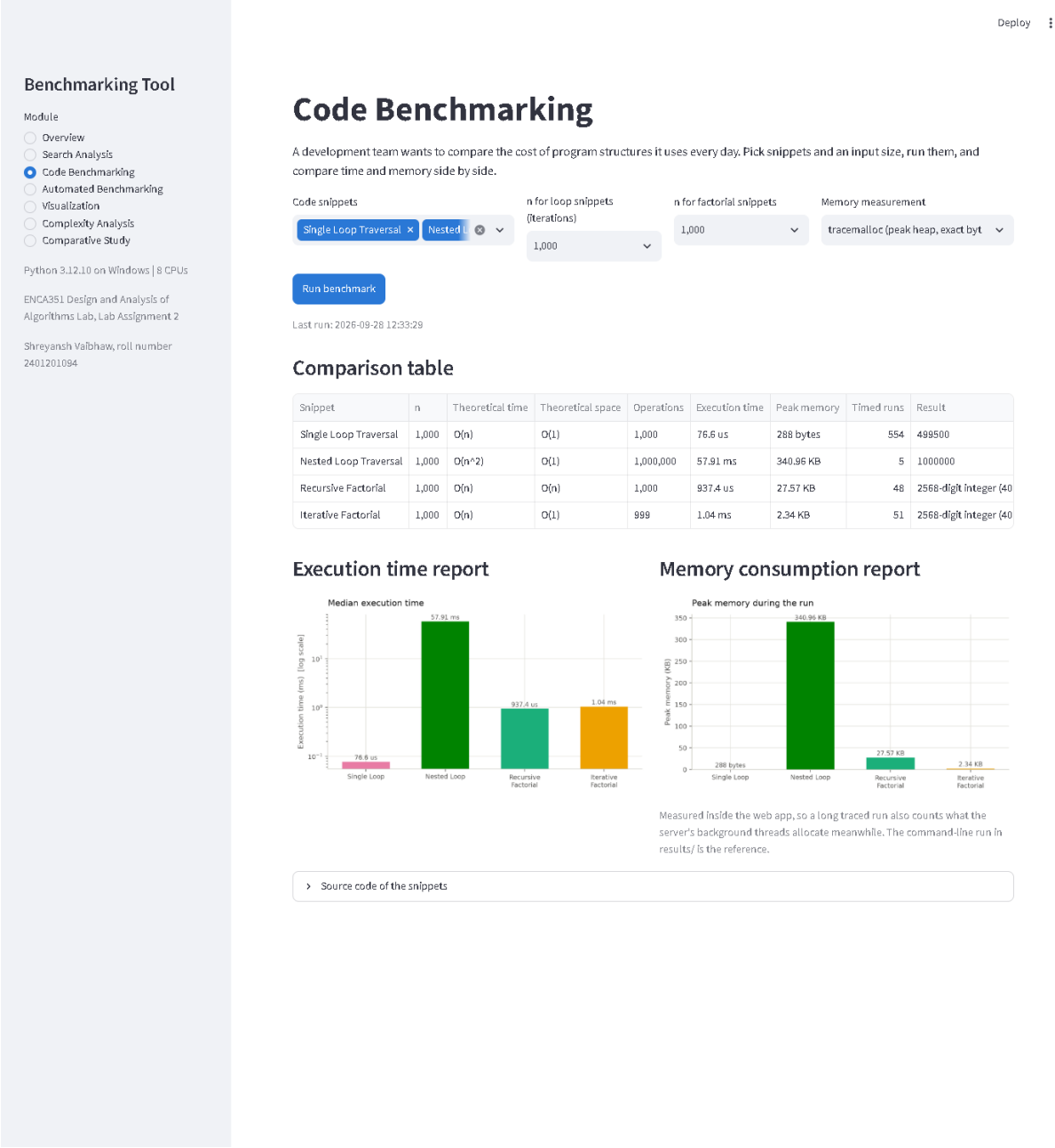
Where to start

1. **Search Analysis:** generate a dataset and search it.
2. **Code Benchmarking:** run the four snippets at one input size.
3. **Automated Benchmarking:** run the whole study (about six minutes with the 50,000-iteration nested loop).
4. **Visualization, Complexity Analysis and Comparative Study** read those results.

Benchmark results are available (22 rows). Loaded from `benchmark_results.csv` (saved 2026-09-28 11:22).

Figure 11. Overview page: what the tool measures and how.





Benchmarking Tool

Module

- ☐ Overview
- ☐ Search Analysis
- ☐ Code Benchmarking
- ☒ Automated Benchmarking
- ☐ Visualization
- ☐ Complexity Analysis
- ☐ Comparative Study

Python 3.12.10 on Windows | 8 CPUs

ENCA351 Design and Analysis of Algorithms Lab, Lab Assignment 2

Shreyansh Vaibhaw, roll number
2401201094

Automated Benchmarking

Runs every algorithm and snippet across the input sizes from the assignment and records execution time, memory consumption and the number of operations for each run.

Showing the saved results from results/benchmark_results.csv. Loaded from benchmark_results.csv (saved 2026-09-28 11:22). Run a new benchmark to replace them.

Input sizes

- ☒ Search algorithms
 ☒ Factorial programs
 ☒ Loop-based programs

Dataset sizes

100, 1000, 10000, 50000

Values of n

100,500,1000

Iteration counts

100, 1000, 10000, 50000

Options

- ☒ Measure memory in a separate tracemalloc... ☐ Skip the memory pass if tracing is predicted to exceed (seconds) ☒ Save results to results/benchmark_results.csv

30

The nested loop at $n = 50,000$ alone takes about 180.15 s on this machine. The page stays busy until the whole run finishes.

Loaded from benchmark_results.csv (saved 2026-09-28 11:22).

Recorded measurements

Algorithm	Input size (n)	Execution time	Peak memory	Operations	Timed runs	Memory method
Linear Search	100	10.5 us	288 bytes	100	1000	tracemalloc (peak heap)
Linear Search	1,000	196.7 us	376 bytes	1,000	222	tracemalloc (peak heap)
Linear Search	10,000	1.66 ms	376 bytes	10,000	25	tracemalloc (peak heap)
Linear Search	50,000	10.18 ms	376 bytes	50,000	5	tracemalloc (peak heap)
Binary Search	100	3.5 us	288 bytes	6	1000	tracemalloc (peak heap)
Binary Search	1,000	3.4 us	316 bytes	9	1000	tracemalloc (peak heap)
Binary Search	10,000	4.9 us	316 bytes	13	1000	tracemalloc (peak heap)
Binary Search	50,000	5.0 us	316 bytes	15	1000	tracemalloc (peak heap)
Recursive Factorial	100	21.0 us	324 bytes	100	1000	tracemalloc (peak heap)
Recursive Factorial	500	228.2 us	8.14 KB	500	199	tracemalloc (peak heap)

Pivot views (algorithm x input size)

Execution time (ms)	Peak memory (KB)	Operations
---------------------	------------------	------------

algorithm	100	500	1000	10000	50000
Binary Search	0.0036	None	0.0034	0.0049	0.0049
Iterative Factorial	0.0143	0.1609	0.5585	None	None
Linear Search	0.0105	None	0.1967	1.5607	10.101
Nested Loop Traversal	0.6911	None	80.2139	8847.8325	328991.85
Recursive Factorial	0.021	0.2282	0.592	None	None
Single Loop Traversal	0.007	None	0.0784	0.8562	4.44

[Download results as CSV](#)

Figure 14. Automated benchmarking page: input sizes, options and the recorded measurements.

Benchmarking Tool

Module

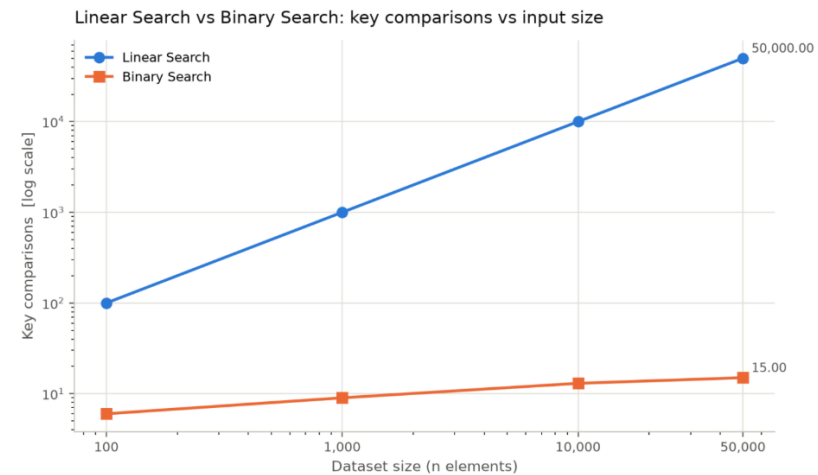
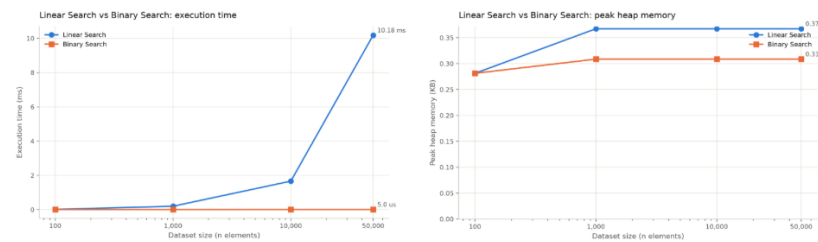
- ☐ Overview
- ☐ Search Analysis
- ☐ Code Benchmarking
- ☐ Automated Benchmarking
- ☒ Visualization
- ☐ Complexity Analysis
- ☐ Comparative Study

Python 3.12.10 on Windows | 8 CPUs

ENCA351 Design and Analysis of
Algorithms Lab, Lab Assignment 2Shreyansh Valbhav, roll number
2401201094

Performance Visualization

Loaded from benchmark_results.csv (saved 2026-09-28 11:22).

☐ Logarithmic time axis (shows both lines when one is thousands of times faster)[Search algorithms](#) Factorial programs Loop-based programs Overview

What the measurements show

- Linear Search: input grew 500x from 100 to 50,000; time grew 969.6x from 10.5 us to 10.18 ms (log-log slope 1.09).
- Binary Search: input grew 500x from 100 to 50,000; time grew 1.4x from 3.6 us to 5.0 us (log-log slope 0.06).
- At $n = 50,000$, Binary Search is 2,036.1x faster than Linear Search.

[Save all charts to graphs/](#)

Figure 15. Visualization page: charts and trend summaries per comparison.

Benchmarking Tool

Module

- ☐ Overview
- ☐ Search Analysis
- ☐ Code Benchmarking
- ☐ Automated Benchmarking
- ☐ Visualization
- ☒ Complexity Analysis
- ☐ Comparative Study

Python 3.12.1.0 on Windows [8 CPUs]

ENCA351 Design and Analysis of Algorithms Lab, Lab Assignment 2

Shreyansh Valbhav, roll number 2401201094

Deploy 

Complexity Analysis

Theoretical complexity

Algorithm / Program	Best Case	Average Case	Worst Case	Time Complexity	Space Complexity	Note
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(1)$	Examines every element when the key is absent.
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(1)$	Needs sorted input; halves the interval each probe.
Recursive Factorial	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	n stack frames; the default recursion limit (1000).
Iterative Factorial	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(1)$	One accumulator; no extra frames.
Single Loop Traversal	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(1)$	One pass over n items.
Nested Loop Traversal	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	$n \times n$ inner iterations.

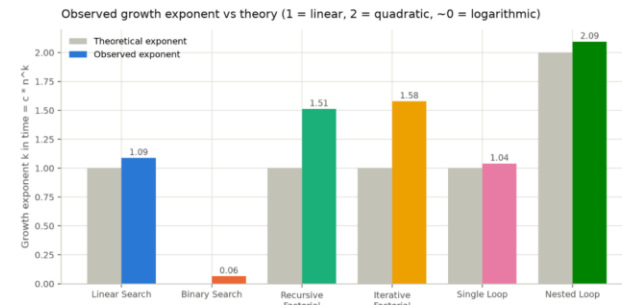
Loaded from benchmark_results.csv (saved 2026-09-28 11:22).

Theoretical vs observed performance

Algorithm / Program	Theoretical time	Theoretical space	Time at smallest n	Time at largest n	Time growth	Peak memory (min to max)	Obs.
Linear Search	$O(n)$	$O(1)$	10.5 us ($n=100$)	10.18 ms ($n=50,000$)	969.6x	288 bytes to 376 bytes	1.0
Binary Search	$O(\log n)$	$O(1)$	3.6 us ($n=100$)	5.0 us ($n=50,000$)	1.4x	288 bytes to 316 bytes	0.0
Recursive Factorial	$O(n)$	$O(n)$	21.0 us ($n=100$)	692.0 us ($n=1,000$)	33.0x	324 bytes to 23.77 KB	1.5
Iterative Factorial	$O(n)$	$O(1)$	14.3 us ($n=100$)	558.5 us ($n=1,000$)	39.2x	324 bytes to 2.39 KB	1.5
Single Loop Traversal	$O(n)$	$O(1)$	7.0 us ($n=100$)	4.44 ms ($n=50,000$)	634.6x	256 bytes to 292 bytes	1.0
Nested Loop Traversal	$O(n^2)$	$O(1)$	691.1 us ($n=100$)	328.89 s ($n=50,000$)	476,040.8x	296 bytes to 320 bytes	2.0

Time relative to the smallest input

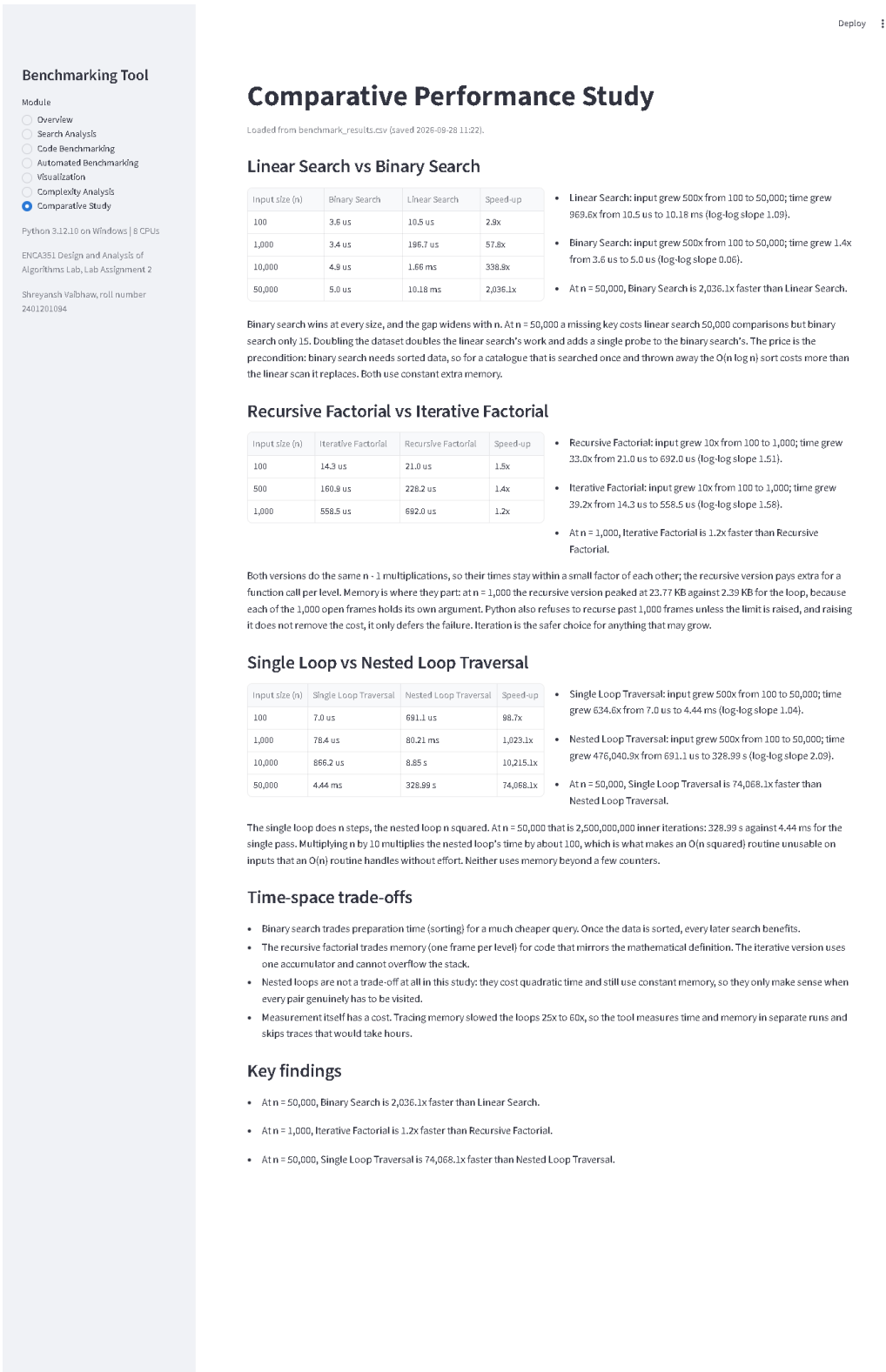
algorithm	100	500	1000	10000	50000
Linear Search	1	None	16.7	158.2	969.6
Binary Search	1	None	0.8	1.4	1.4
Recursive Factorial	1	10.9	39	None	None
Iterative Factorial	1	11.3	39.2	None	None
Single Loop Traversal	1	None	11.2	123.7	634.6
Nested Loop Traversal	1	None	116.1	12802.5	476040.9



Reading the table

- The observed exponent is the slope of $\log(\text{time})$ against $\log(n)$. It is fitted from the measured points, so with only three or four sizes it is an estimate, not a proof.
- Binary search runs in a few microseconds at every size. At that scale the fixed cost of a function call and the timer itself is a large share of the measurement, which is why its line is nearly flat rather than a clean logarithm.
- Both factorial versions grow faster than $O(n)$ because Python integers have arbitrary precision: 1000! has 2,568 digits, and multiplying a number that long is not a constant-time step. The $O(n)$ bound counts multiplications, not the digits they touch.
- tracemalloc sees Python heap allocations. The recursive factorial shows $O(n)$ growth because every open call keeps its own integer argument alive, one per stack frame.
- The nested loop's memory pass is skipped at large n because tracing every allocation would take hours; its $O(1)$ memory is verified at the smaller sizes where the trace is affordable.

Figure 16. Complexity analysis page: theoretical against observed performance.



4. Analysis

4.1 Comparison of search algorithms

Binary search was faster at every size, and the ratio grew with n : 2.9x at 100 elements and 2,036x at 50,000. The comparison counts explain it. Linear search examined every element, 50,000 comparisons for an absent key at 50,000, while binary search needed 15 probes, 2 more than at 10,000. Doubling the catalogue doubles the linear search's work and adds a single probe to the binary search's.

The observed exponents were 1.09 for linear search and 0.06 for binary search. Linear search matches its $O(n)$ bound closely. Binary search's line is almost flat rather than a clean logarithm because a five-microsecond call is dominated by fixed costs: entering the function, setting up the loop, and the timer itself. The logarithm is visible in the comparison counts (6, 9, 13, 15) rather than in the clock.

Both searches used constant memory, about 300 to 380 bytes, which is the loop variables and the result object. What binary search asks in return is sorted input. Sorting costs $O(n \log n)$ once. For a catalogue that is searched a handful of times, a linear scan can be the cheaper total; for a library catalogue queried thousands of times a day, the sort is paid once and every lookup benefits.

4.2 Comparison of recursive and iterative implementations

Both factorial versions perform the same $n - 1$ multiplications, so their times stayed within a small factor: at $n = 1,000$ the recursive version took 1.24x as long as the loop, the extra being one function call per level. Memory is where they part. At $n = 1,000$ the recursive version peaked at 23.77 KB, the loop at 2.39 KB, and the recursive figure grows linearly with n (Figure 6). Every open recursive call keeps its own integer argument alive on the heap, one object per frame. The interpreter frames themselves are not visible to `tracemalloc`, so the true stack cost is higher still.

Python enforces that cost with a hard limit: it refuses to recurse past 1,000 frames by default, so $n = 1,000$ could not run until the tool raised `sys.setrecursionlimit`. Raising the limit does not remove the cost, it only moves the point of failure. The loop keeps one accumulator, cannot overflow the stack, and was faster at every size. For anything whose input may grow, iteration is the safer choice.

One result surprised me. Both factorials grew faster than their $O(n)$ bound predicts, with exponents of 1.51 and 1.58. The reason is arithmetic, not control flow: $1000!$ has 2,568 digits, and multiplying a number that long by a small integer takes time proportional to its length. The $O(n)$ analysis assumes constant-time multiplication, which holds for machine integers and fails for Python's arbitrary-precision ones. Measured across the three sizes the cost looks like n to the power 1.5, which is what n multiplications of numbers that grow with n produce.

4.3 Complexity analysis

Table 4 sets the theoretical complexity against what was measured. The observed exponent is the slope of a straight-line fit to $\log(\text{time})$ against $\log(n)$ across the measured sizes; slope 1 is linear growth, slope 2 quadratic, and a slope near 0 constant or logarithmic.

Algorithm	Theory: time	Theory: space	Time at smallest n	Time at largest n	Growth	Exponent	Observed trend	Verdict
Linear Search	$O(n)$	$O(1)$	10.5 us	10.18 ms	970x	1.09	linear	matches theory
Binary Search	$O(\log n)$	$O(1)$	3.6 us	5.0 us	1x	0.06	constant / logarithmic	matches theory
Recursive Factorial	$O(n)$	$O(n)$	21.0 us	692.0 us	33x	1.51	between linear and quadratic ($n \log n$ -like)	grows faster than the bound predicts
Iterative Factorial	$O(n)$	$O(1)$	14.3 us	558.5 us	39x	1.58	between linear and quadratic ($n \log n$ -like)	grows faster than the bound predicts
Single Loop Traversal	$O(n)$	$O(1)$	7.0 us	4.44 ms	635x	1.04	linear	matches theory
Nested Loop Traversal	$O(n^2)$	$O(1)$	691.1 us	328.99 s	476,041x	2.09	quadratic	matches theory

Table 4. Theoretical against observed performance.

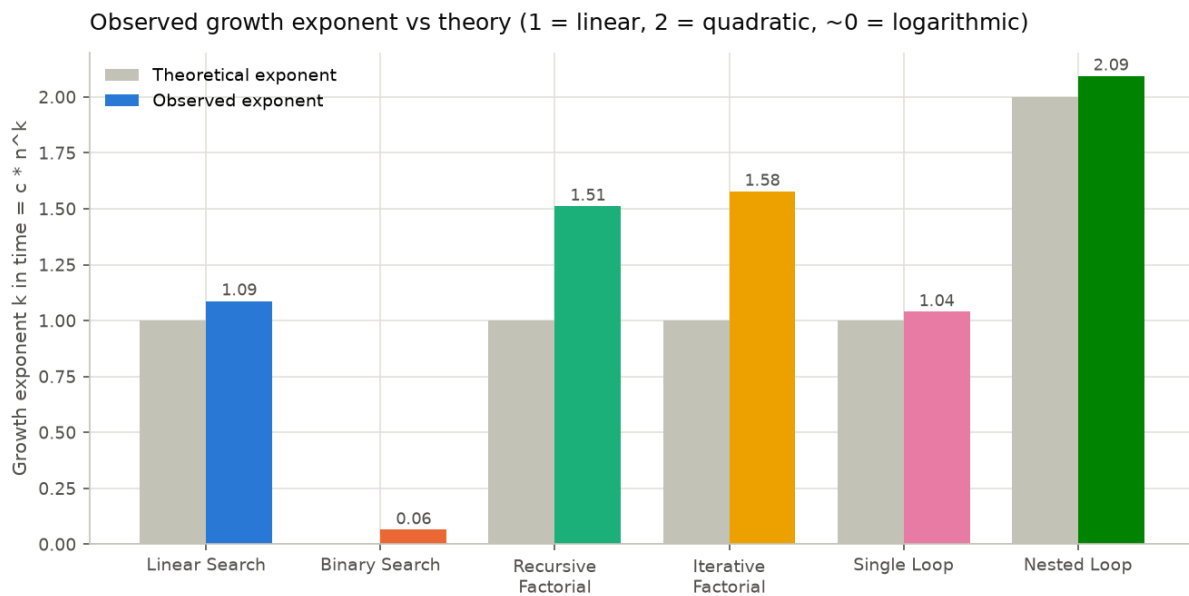


Figure 18. Observed growth exponent next to the exponent theory predicts.

Linear search (1.09), the single loop (1.04) and the nested loop (2.09) land close to their predicted exponents of 1, 1 and 2. Binary search (0.06) is in the constant-or-logarithmic band, as expected. The two factorials sit between linear and quadratic for the reason given in section 4.2. Space complexity was confirmed directly: constant memory for the searches and loops, linear growth for the recursive factorial.

4.4 Performance observations

- **Impact of input size.** A 500x increase in input multiplied the linear search's time by 970x, the single loop's by 635x, the nested loop's by 476,041x, and the binary search's by 1.4x. Order of growth, not the constant factor, decides what happens at scale.

- **Complexity against actual time.** Big-O predicted the ranking correctly in every comparison, but constants still matter within a class. Linear search and the single loop are both $O(n)$, yet at $n = 50,000$ the search took 10.18 ms and the loop 4.44 ms: enumerating and comparing each element costs more than adding it.
- **Time-space trade-offs.** Binary search trades preparation time (sorting) for cheap queries. The recursive factorial trades memory, one frame per level, for code that mirrors the mathematical definition. Nested loops are not a trade-off at all: they cost quadratic time and still only use a few counters, so they are only justified when every pair genuinely has to be visited.
- **Measurement has a cost.** Tracing memory slowed the loops 24x to 60x, which is why the engine measures time and memory in separate runs and skips traces that would take hours. At the microsecond scale the timer and the call overhead are a large share of what is measured, which is why binary search's curve is flatter than a logarithm.

5. Conclusion

5.1 Key findings

- Binary search beat linear search by 2,036x at 50,000 elements, and the gap doubles whenever the dataset doubles.
- The nested loop needed 328.99 s at 50,000 iterations against 4.44 ms for the single loop. An $O(n^2)$ routine is already impractical at a size an $O(n)$ routine handles in milliseconds.
- The recursive and iterative factorials ran within 1.24x of each other, but the recursive version's memory grew linearly with n and needed the recursion limit raised to reach $n = 1,000$.
- Both factorials grew faster than $O(n)$ because Python's arbitrary-precision multiplication is not a constant-time step. Big-O is only as good as its assumptions.
- Careful measurement matters: separating timing from tracing, repeating fast calls, excluding input generation and pausing garbage collection all changed the numbers noticeably during development.

5.2 Most efficient implementations observed

- Search: binary search, provided the data is sorted and searched more than a few times.
- Factorial: the iterative version, faster at every size, constant frames, no recursion limit.
- Traversal: the single loop; the nested loop should only be used when every pair must be visited.

5.3 Future enhancements

- Add sorting and hashing algorithms, and a hash-table lookup to compare with binary search.
- Report confidence intervals from the repeated runs instead of a single median.
- Measure the interpreter's frame stack directly, for example by sampling stack depth, so recursion cost is not inferred from heap objects.

- Let users paste their own snippet, run it in a sandboxed subprocess and benchmark it alongside the built-in ones.
- Compare interpreters (CPython versions, PyPy) to show how much of a constant factor is the runtime rather than the algorithm.

References

- Horowitz, E., Sahni, S. and Rajasekaran, S. Fundamentals of Computer Algorithms. Universities Press.
- Kleinberg, J. and Tardos, E. Algorithm Design. Pearson.
- Python Software Foundation. The Python Standard Library: time, tracemalloc, timeit, sys. <https://docs.python.org/3/library/>
- Streamlit Inc. Streamlit documentation. <https://docs.streamlit.io>
- Pedregosa, F. et al. memory_profiler. https://github.com/pythonprofilers/memory_profiler
- Project Jupyter. Jupyter Notebook documentation. <https://jupyter-notebook.readthedocs.io>
- Hunter, J. D. Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 2007.

Appendix A. Complete measurement table

Every row recorded by the reference run (results/benchmark_results.csv). Time is the median; min and max are the fastest and slowest of the timed runs.

Algorithm	n	Median time	Min	Max	Runs	Peak memory	Operations
Linear Search	100	10.5 us	10.0 us	81.1 us	1000	288 bytes	100
Linear Search	1,000	196.7 us	134.2 us	1.23 ms	222	376 bytes	1,000
Linear Search	10,000	1.66 ms	1.57 ms	4.72 ms	25	376 bytes	10,000
Linear Search	50,000	10.18 ms	8.76 ms	10.99 ms	5	376 bytes	50,000
Binary Search	100	3.6 us	2.3 us	42.1 us	1000	288 bytes	6
Binary Search	1,000	3.4 us	3.3 us	18.9 us	1000	316 bytes	9
Binary Search	10,000	4.9 us	4.6 us	64.8 us	1000	316 bytes	13
Binary Search	50,000	5.0 us	4.9 us	138.9 us	1000	316 bytes	15
Recursive Factorial	100	21.0 us	20.4 us	176.2 us	1000	324 bytes	100
Recursive Factorial	500	228.2 us	217.7 us	806.8 us	199	8.14 KB	500
Recursive Factorial	1,000	692.0 us	666.2 us	1.16 ms	67	23.77 KB	1,000
Iterative Factorial	100	14.3 us	13.5 us	77.5 us	1000	324 bytes	99
Iterative Factorial	500	160.9 us	130.2 us	299.4 us	296	1.15 KB	499
Iterative Factorial	1,000	558.5 us	471.7 us	971.7 us	87	2.39 KB	999
Single Loop Traversal	100	7.0 us	6.5 us	103.7 us	1000	256 bytes	100
Single Loop Traversal	1,000	78.4 us	76.1 us	246.2 us	583	288 bytes	1,000
Single Loop Traversal	10,000	866.2 us	813.8 us	1.70 ms	52	288 bytes	10,000
Single Loop Traversal	50,000	4.44 ms	4.21 ms	5.31 ms	12	292 bytes	50,000
Nested Loop Traversal	100	691.1 us	630.0 us	1.08 ms	69	256 bytes	10,000
Nested Loop Traversal	1,000	80.21 ms	77.51 ms	90.30 ms	5	320 bytes	1,000,000
Nested Loop Traversal	10,000	8.85 s	8.85 s	8.85 s	1	not traced	100,000,000
Nested Loop Traversal	50,000	328.99 s	328.99 s	328.99 s	1	not traced	2,500,000,000